

Physical Inconsistency Detector

End-to-End Report on Residual Probes, Multi-View Validation, Temporal Drift
Detection, Semantic Reasoning, and Counterfactual Data Repair

Diksha Jangam
University of Birmingham

June 9, 2026

Contents

1	Abstract	4
2	Project Motivation and Objectives	5
3	Overall System Overview	5
4	Phase 1: Environment and Synthetic Data Preparation	6
4.1	Repository and Environment Setup	6
4.2	Initial Synthetic Single-View Dataset	7
4.3	Single-View Corruption Strategy	7
5	Phase 2: DINOv2 Feature Extraction	7
5.1	Single-View Embedding Extraction	7
5.2	Why Frozen Features Were Used	8
6	Phase 3: Transition Probe Modeling	8
6.1	Baseline Transition Probe	8
6.2	Correct Training Protocol	8
7	Phase 4: Single-View Residual Scoring and Calibration	8
7.1	Latent Residual Score	8
7.2	Single-View Performance Summary	8
7.3	Platt Scaling	9
8	Phase 5: Single-View Reliability and Ranking Metrics	9
8.1	Calibration and Reliability	9
8.2	Ranking Metrics	9
8.3	Lift@k	9
8.4	Bootstrap Confidence Intervals	10
9	Visualization Layer: Live Rerun Inspection	10
9.1	Rerun-Based Inspection Interface	10
10	Multi-View Extension	10
10.1	Motivation	10
10.2	Multi-View Synthetic Dataset	11
10.3	True Spatial Occlusion Anomaly	11
10.4	Multi-View Feature Representation	11
10.5	Multi-View Probe	11
10.6	Multi-View Results	11
11	Temporal Hardware / Visual Synchronization Layer	12
11.1	Mock Hardware Telemetry	12
11.2	Injected Temporal Faults	12
11.3	Temporal Validation Function	13

11.4 Timing Fault Routing	13
12 Counterfactual Intervention Engine	13
12.1 Motivation	13
12.2 Intervention Modes	13
12.3 Integration into Main Loop	13
12.4 Remedy Summaries	14
13 Semantic Scene-Graph Validation	14
13.1 Semantic Reasoning Layer	14
13.2 Expected vs Observed Graphs	14
13.3 Injected Semantic Anomaly	14
13.4 Semantic Discrepancy Signal	14
13.5 Semantic Override	15
14 Repository Reorganization	15
15 Downstream Ablation Study: Evaluating the Value of Lift	15
15.1 Goal	15
15.2 Downstream Model	15
15.3 Ablation Configurations	16
15.4 Downstream Results	16
15.5 Interpretation	16
15.6 Value of Lift Chart	16
16 Discussion	17
16.1 What Worked Well	17
16.2 What Was More Difficult	17
17 Limitations	17
18 Conclusion	18
19 Future Work	18

1 Abstract

This report documents the complete design, implementation, and evaluation of a synthetic **Physical Inconsistency Detector** pipeline. The system was first built as a single-view residual-based anomaly detector using frozen DINOv2 embeddings and a lightweight transition-prediction probe. The original goal was to detect physically broken state transitions in a small synthetic video dataset. The project was then expanded with several additional capabilities: a multi-view representation, temporal hardware/vision synchronization monitoring, a counterfactual intervention engine, a semantic scene-graph validation module, and a downstream ablation study that measured the practical value of uncertainty routing and synthetic correction.

The first system stage used synthetic square-motion clips, extracted first/last frame embeddings using a frozen DINOv2 encoder, trained a transition probe on clean trajectories only, and computed a latent residual score at inference time. Residuals were converted into calibrated probabilities using logistic regression (Platt scaling), after which standard reliability and ranking metrics were computed. The single-view system achieved near-perfect separation between clean and corrupted examples on the original synthetic dataset.

The second stage extended the project into a multi-view formulation by introducing synchronized overhead and egocentric camera streams. Both views were passed through DINOv2 and concatenated into a multi-view state descriptor. A stronger test anomaly was created by applying ego-view occlusion while keeping the overhead stream intact. This produced a measurable spatial discrepancy signal and allowed multi-view residual analysis under visibility mismatch.

The third stage added temporal hardware validation. A mock hardware joint-velocity telemetry stream was created and aligned with the visual timeline. Two temporal fault modes were injected: timing drift and dropped-frame freezes. A rolling mismatch signal between visual motion and hardware telemetry was computed and used to raise timing fault alarms.

The fourth stage added an intervention engine capable of generating counterfactual synthetic repairs of anomaly frames. Different correction modes were triggered depending on whether the failure was primarily spatial, temporal, or combined. These interventions were logged back into the live visualization dashboard.

The fifth stage introduced semantic reasoning. A small rule-based scene-graph layer compared expected semantic relations (derived from action claims) against observed relations inferred from image-space object location. This enabled structural contradiction detection, such as an action claim stating that a box was placed into a bin while the observed box remained on the table.

Finally, a downstream ablation study tested whether the uncertainty-routing and intervention pipeline produced value for a separate box-detection model. Four training configurations were compared: clean-only training, noisy raw-data augmentation, naive filtering, and counterfactual synthetic recovery. The results showed that intervention-based recovery improved average localization quality, while naive deletion of uncertain samples performed worst under thresholded detection accuracy.

Overall, this project demonstrates a complete synthetic research prototype for *self-healing data loops*, combining latent residual probes, calibration, multi-view consistency, temporal synchronization checks, semantic mismatch validation, and counterfactual repair into a single end-to-end analysis pipeline.

2 Project Motivation and Objectives

The core motivation of this project was to build a compact but complete system for detecting physically inconsistent transitions in synthetic motion sequences. The original design brief targeted a workflow that starts with synthetic timeline generation, extracts fixed-image embeddings, trains a lightweight transition model, scores transition residuals, calibrates the scores, and reports reliability metrics in a polished repository. The project phases called for environment setup, mock data generation, DINOv2 feature extraction, a transition probe, residual scoring, calibration, Lift@k analysis, bootstrap confidence intervals, and repository packaging. The original brief also emphasized the use of a lightweight probe trained on clean data only and the conversion of raw residual distances into calibrated failure probabilities.

The project later evolved from a basic latent residual detector into a broader synthetic data-engine research prototype with the following extended goals:

1. Detect broken physical transitions using latent residuals.
2. Route uncertain samples to human review using calibrated probabilities and Lift@k.
3. Expand the detector into a multi-view spatial validation system.
4. Add temporal cross-checking between visual motion and mock hardware telemetry.
5. Generate automated counterfactual corrective variations for flagged samples.
6. Introduce basic semantic scene-graph reasoning to detect structural contradictions.
7. Evaluate whether uncertainty-aware filtering or synthetic repair improves downstream learning.

The final system therefore serves as a small but realistic demonstration of a closed-loop data curation and quality-control pipeline.

3 Overall System Overview

At a high level, the completed project consists of the following components:

1. **Synthetic Data Generation:** create short video-like sequences of a moving square.
2. **Corruption / Anomaly Injection:** inject known anomalies into a subset of the data.
3. **Feature Extraction:** encode visual states using frozen DINOv2 embeddings.
4. **Transition Probe:** predict the next-state latent embedding from a current-state latent embedding and action cue.
5. **Residual Scoring:** compute the latent error between predicted and actual state.
6. **Calibration:** convert residual distances into calibrated probabilities using logistic regression.
7. **Evaluation Metrics:** reliability diagrams, Brier score, AUROC, AUPRC, Lift@k, and bootstrap confidence intervals.

8. **Multi-View Extension:** combine overhead and egocentric embeddings into a larger spatial descriptor.
9. **Temporal Telemetry Validation:** compare visual motion with mock hardware joint velocity.
10. **Intervention Engine:** automatically generate corrected synthetic variations for flagged samples.
11. **Semantic Validation:** compare action claims against observed scene-graph relations.
12. **Downstream Ablation:** measure the value of uncertainty handling and repair on a separate detector.

This final architecture supports multiple forms of evidence:

- latent residual evidence,
- spatial multi-view evidence,
- temporal synchronization evidence,
- semantic relation evidence,
- and intervention outputs.

4 Phase 1: Environment and Synthetic Data Preparation

4.1 Repository and Environment Setup

The project began by creating a fresh Git repository inside the WSL workspace and preparing a Python virtual environment. The required stack included:

- `torch`
- `transformers`
- `opencv-python`
- `scikit-learn`
- `matplotlib`

These packages were used for:

- neural network training,
- frozen-image embedding extraction,
- image and synthetic video manipulation,
- calibration and metrics,
- and plotting.

4.2 Initial Synthetic Single-View Dataset

The original dataset consisted of 100 small synthetic “video” folders, each containing 10 frames. The frames were generated as black background images with a moving white square. The square motion followed simple deterministic paths. This was used as a minimal synthetic physical world.

The initial single-view data generator produced folders of the form:

```
data/raw/video_0/frame_0.png
data/raw/video_0/frame_1.png
...
data/raw/video_99/frame_9.png
```

4.3 Single-View Corruption Strategy

The first corruption strategy modified 20% of videos by replacing the final frame. The early version of the corruption was relatively weak. Later, stronger corruptions were introduced, including replacement by random-noise images. The system also tracked exactly which videos were corrupted by storing:

```
data/corrupted_list.txt
```

This enabled a clean split:

- 80% clean for training,
- 20% corrupted for evaluation.

5 Phase 2: DINOv2 Feature Extraction

5.1 Single-View Embedding Extraction

The original feature-extraction pipeline used a frozen DINOv2 model:

```
facebook/dinov2-base
```

For each video:

- the first frame was mapped to a latent vector z_{before} ,
- the last frame was mapped to a latent vector z_{after} .

The extractor used the CLS token from the last hidden state as the global image representation.

The output was saved in compressed files:

```
data/processed_features/video_0.npz
```

Each file stored:

- before
- after

5.2 Why Frozen Features Were Used

Using frozen DINOv2 features made the system much lighter than training a full visual model from scratch. It also ensured that:

- feature extraction happened once,
- the expensive encoder did not need to be repeatedly re-run,
- the downstream transition probe could be trained quickly.

6 Phase 3: Transition Probe Modeling

6.1 Baseline Transition Probe

The first transition probe was a lightweight MLP:

$$f_{\theta}(z_{\text{before}}, a) \approx \hat{z}_{\text{after}}$$

where:

- z_{before} = DINOv2 embedding of the first frame,
- a = one-hot action vector,
- $\hat{z}_{\text{after}}$ = predicted next-state embedding.

The probe was trained using mean squared error (MSE):

$$\mathcal{L}_{\text{MSE}} = \|z_{\text{after}} - \hat{z}_{\text{after}}\|_2^2$$

6.2 Correct Training Protocol

An important correction was made during development: the model was retrained using only the clean 80% data. The corrupted videos were removed from training and reserved for testing. This ensured that the model learned normal transitions only.

7 Phase 4: Single-View Residual Scoring and Calibration

7.1 Latent Residual Score

At inference time, the system compared the predicted state embedding with the actual final-state embedding:

$$\text{Residual} = \|z_{\text{after}} - \hat{z}_{\text{after}}\|_2$$

This produced an anomaly score for each sample.

7.2 Single-View Performance Summary

The final single-view run produced the following results:

Table 1: Single-View Residual Summary

Metric	Clean	Corrupted
Mean Residual	9.3882	59.5817

The top 10 highest-residual samples were all corrupted, which showed that the residual score separated the two classes extremely well.

7.3 Platt Scaling

Residuals were then transformed into calibrated probabilities using a logistic regression layer:

$$p(y = 1 \mid r) = \sigma(wr + b)$$

where:

- r is the residual,
- $y = 1$ denotes physical failure.

The calibrated probabilities for top anomalous samples were near 1.0 in the single-view system.

8 Phase 5: Single-View Reliability and Ranking Metrics

8.1 Calibration and Reliability

The reliability evaluation produced a very small Brier score:

$$\text{Brier Score} = 0.000001$$

This indicates that the calibrated probabilities were extremely accurate on the synthetic single-view benchmark.

8.2 Ranking Metrics

The single-view system also achieved:

- AUROC = 1.000000
- AUPRC = 1.000000

These values indicate perfect class separation on the original synthetic benchmark.

8.3 Lift@k

Lift@k analysis showed:

Table 2: Single-View Lift@k

Top Fraction	Corrupted Found	Positive Rate	Lift@k
Top 10%	10/10	1.0000	5.0000
Top 20%	20/20	1.0000	5.0000
Top 30%	20/30	0.6667	3.3333

This means that reviewing only the highest-risk samples was much more efficient than random review.

8.4 Bootstrap Confidence Intervals

Bootstrap evaluation on the single-view system produced:

- Brier Score mean = 0.000001, 95% CI = [0.000000, 0.000003]
- AUROC mean = 1.000000, 95% CI = [1.000000, 1.000000]
- AUPRC mean = 1.000000, 95% CI = [1.000000, 1.000000]

These intervals were extremely tight, which is consistent with the synthetic benchmark being easy to separate.

9 Visualization Layer: Live Rerun Inspection

9.1 Rerun-Based Inspection Interface

An interactive inspection interface was added using Rerun. Because of WSL graphics limitations, the implementation was adapted to use a running Rerun viewer connection instead of the simplest native spawn route. The visualization script logged:

- the before/after visual states,
- latent residual,
- calibrated uncertainty,
- routing status messages,
- and later additional telemetry and intervention views.

This interface became the central live debugging and explanation surface for the rest of the project.

10 Multi-View Extension

10.1 Motivation

The single-view detector worked very well, but it only reasoned about one view. To make the anomaly detection problem richer, the system was extended to use:

- an **overhead** view,
- and a **robot ego** view.

10.2 Multi-View Synthetic Dataset

A new multi-view dataset was created with folders of the form:

```
data/multiview/raw/video_0/overhead/frame_0.png
data/multiview/raw/video_0/ego/frame_0.png
```

The box motion was rendered synchronously in both views, but the ego image used a second deterministic mapping from the same latent state.

10.3 True Spatial Occlusion Anomaly

A new corruption script created a real multi-view visibility mismatch:

- the final **overhead** frame remained valid,
- the final **ego** frame was occluded with a large black block.

This created a physically meaningful discrepancy:

$$\text{visible in overhead} \quad \wedge \quad \text{blocked in ego}$$

10.4 Multi-View Feature Representation

Both images were passed through the frozen DINOv2 encoder:

$$z_{\text{overhead}}, \quad z_{\text{ego}}$$

The final multi-view descriptor was:

$$z_{\text{spatial}} = [z_{\text{overhead}}; z_{\text{ego}}]$$

10.5 Multi-View Probe

The transition probe was updated to accept:

$$[z_{\text{spatial}}; a]$$

and predict:

$$\hat{z}_{\text{spatial, after}}$$

This increased both the input and output dimensionality of the transition model.

10.6 Multi-View Results

The multi-view evaluation produced the following summary:

Table 3: Multi-View Residual Summary

Metric	Clean	Corrupted
Mean Residual	12.3996	23.5409
Mean Spatial Anomaly	2.6232	13.3313

The multi-view ranking and calibration metrics were:

- AUROC = 0.942500
- AUPRC = 0.790215
- Brier Score = 0.084831

This demonstrates that the multi-view system still clearly separated clean and corrupted samples, while solving a harder visibility-mismatch problem than the original single-view benchmark.

11 Temporal Hardware / Visual Synchronization Layer

11.1 Mock Hardware Telemetry

A mock hardware telemetry stream was added to the pipeline, representing a joint-velocity-like scalar aligned to the same sequence timeline as the visual samples. This signal was logged into its own Rerun telemetry plot:

```
hardware/joint_velocity
```

11.2 Injected Temporal Faults

Two temporal anomaly modes were introduced:

1. Timing Drift Hardware values were intentionally shifted forward by 5 steps across a window:

- Start = step 20
- End = step 40
- Shift = 5

This simulated telemetry lag/lead relative to video.

2. Dropped Frames A visual freeze region was created:

- Start = step 60
- End = step 72

In this region, the image frames repeated while hardware telemetry continued changing.

11.3 Temporal Validation Function

A rolling mismatch score was computed between:

- visual motion (estimated from object position change),
- hardware motion (mock joint velocity).

This produced a scalar:

`temporal_sync_drift`

It was logged in Rerun as:

```
evidence/temporal_sync_drift
```

11.4 Timing Fault Routing

When temporal drift exceeded the threshold or when dropped frames were detected, the system emitted:

`TIMING FAULT: Hardware/Visual Sync Drift Detected`

This created a separate timing-fault route that worked alongside spatial and semantic anomaly routing.

12 Counterfactual Intervention Engine

12.1 Motivation

An intervention engine was added to create synthetic repair proposals for problematic samples. The idea was to simulate an automated “self-healing data loop.”

12.2 Intervention Modes

A separate module `intervention_engine.py` was created with:

- **Gaussian blur** for high spatial anomaly samples,
- **localized gamma/contrast correction** for temporal sync drift samples,
- **combined correction** for cases where both failures were present.

12.3 Integration into Main Loop

When a sample triggered either:

- `[HUMAN REVIEW REQUIRED]`
- or an `ERROR` timing fault,

the final state image was passed into:

```
generate_counterfactual_variation(image, failure_mode)
```

The corrected image was then logged in Rerun as:

```
intervention/proposed_synthetic_variation
```

12.4 Remedy Summaries

The text logs were updated to include messages such as:

```
[video_30] INTERVENTION GENERATED: Counterfactual variation rendered to
correct localized anomaly channel.
```

This created a visible intervention feed synchronized with the anomaly routing system.

13 Semantic Scene-Graph Validation

13.1 Semantic Reasoning Layer

A rule-based semantic engine was added to detect contradictions between:

- the claimed action semantics,
- and the final observed physical state.

A small ontology was defined:

- object types: `box`, `table`, `bin`
- relations: `ON_TOP_OF`, `INSIDE`

13.2 Expected vs Observed Graphs

At each step:

1. an action claim was parsed into an expected graph,
2. the image was translated into an observed graph,
3. the two were compared.

13.3 Injected Semantic Anomaly

A semantic anomaly slice was introduced:

- steps 84–90

Inside this region, the action claim became:

```
Box placed in bin
```

while the image-space state continued to imply that the box remained on the table.

13.4 Semantic Discrepancy Signal

A rolling semantic discrepancy score was computed and logged to:

```
evidence/spatial_semantic_mismatch
```

13.5 Semantic Override

A semantic mismatch could trigger a critical routing override with messages like:

```
[video_85] SEMANTIC MISMATCH: Action claim contradicts physical scene graph
state edges.
```

After simplification of the semantic rules, this mismatch behavior was successfully concentrated into the intended anomaly region.

14 Repository Reorganization

Once the multi-view pipeline became the main working system, the project was reorganized into:

- data/legacy_singleview/
- data/multiview/
- results/singleview/
- results/multiview/

This preserved the original single-view baseline while clearly separating the later system extensions.

15 Downstream Ablation Study: Evaluating the Value of Lift

15.1 Goal

A separate downstream detector was trained to test whether:

- uncertainty-aware filtering,
- or intervention-generated synthetic correction,

provides value for another learning system.

15.2 Downstream Model

A small CNN-based box regressor was trained to predict the box position from the overhead image. The detector was evaluated on a deliberately hard validation set containing:

- heavy shadows,
- motion blur,
- rotation / angle changes.

15.3 Ablation Configurations

Four training configurations were evaluated:

1. **Clean Base Data Only**
2. **Base Data + Corrupted/Unfiltered Data**
3. **Base Data + Naive Filtering**
4. **Base Data + Counterfactual Synthetics**

15.4 Downstream Results

The final downstream results were:

Table 4: Downstream Ablation Results

Training Configuration	BBox Accuracy@0.50	Mean IoU
Clean Base Data Only	0.3750	0.4045
Base + Corrupted/Unfiltered Data	0.5625	0.4471
Base + Naive Filtering	0.3125	0.4326
Base + Counterfactual Synthetics	0.5000	0.4589

15.5 Interpretation

The downstream study produced a mixed but useful result:

- **Naive filtering** performed worst on thresholded detection accuracy.
- **Counterfactual synthetics** produced the best average localization quality (`mean_iou`).
- **Raw corrupted data** unexpectedly achieved the best `bbox_acc50`.

This means the strongest support for the closed-loop synthetic-repair thesis appeared in `mean_iou`, while the raw corrupted baseline remained competitive under the harder thresholded metric. Because the downstream validation set was small and heavily synthetic, these results should be interpreted carefully.

15.6 Value of Lift Chart

A final bar chart was generated in Matplotlib to compare all four conditions:

`results/downstream_value_of_lift.png`

This figure should be included in the final repository and report as one of the main summary visuals.

16 Discussion

16.1 What Worked Well

Several parts of the project worked strongly:

- The single-view residual probe achieved near-perfect synthetic separation.
- Calibration and ranking metrics behaved exactly as expected on the single-view benchmark.
- The multi-view system successfully detected visibility mismatch.
- The temporal layer correctly detected both timing drift and dropped frames.
- The intervention engine ran synchronously and generated interpretable synthetic repairs.
- The semantic mismatch layer eventually behaved correctly after simplification.
- The downstream study showed that deleting uncertain samples outright can be harmful, while intervention-based recovery improved average localization quality.

16.2 What Was More Difficult

Several practical difficulties emerged:

- The original weak corruption strategy was not strong enough.
- Random action assignment made the original probe learning unstable.
- Rerun setup in WSL required adaptation due to graphics limitations.
- The semantic layer initially over-triggered because the synthetic world was too simple for a richer graph.
- The downstream detector results did not perfectly match the ideal thesis under every metric.

These issues were all valuable because they improved the design of the final prototype.

17 Limitations

This project is still a synthetic research prototype and has several limitations:

1. The visual world is simple (moving square on black background).
2. The hardware signal is synthetic rather than real telemetry.
3. The semantic engine is rule-based and deliberately simplified.
4. The intervention engine produces heuristic modifications rather than learned repairs.
5. The downstream detector is small and was tested on a small hard-validation split.

6. Several metrics are likely optimistic because the tasks remain highly structured and synthetic.

Therefore, this system should be viewed as a proof-of-concept for data quality loops rather than a production-ready robotic anomaly detector.

18 Conclusion

This project successfully evolved from a simple synthetic latent residual detector into a much larger end-to-end data integrity and intervention framework. The final system includes:

- single-view latent residual scoring,
- multi-view visibility reasoning,
- temporal hardware synchronization monitoring,
- counterfactual repair generation,
- semantic scene-graph validation,
- and downstream value testing.

The strongest result from the early system was the perfect single-view synthetic separation. The strongest result from the later system was the successful integration of multiple evidence channels into a single review-routing interface. The downstream ablation study further suggested that uncertainty-aware repair can improve localization quality better than naive sample deletion.

In summary, the project demonstrates the core idea of a **self-healing data loop**: detect difficult or broken samples, explain why they are risky, surface them for human review, and optionally generate repair proposals that can be fed back into downstream learning.

19 Future Work

Possible next steps include:

1. Replace the synthetic square world with a richer object simulation.
2. Train a learned intervention network rather than using hand-designed image transforms.
3. Replace the rule-based semantic engine with a learned graph-prediction model.
4. Introduce real sequence models instead of first/last frame probes.
5. Use a larger and more realistic downstream detector benchmark.
6. Add real hardware telemetry and real camera calibration.
7. Evaluate intervention fidelity and semantic consistency under larger-scale conditions.

Appendix A: Key Files Produced

File	Purpose
src/extract_features.py	DINOv2 feature extraction (single-view and later multi-view version)
src/train_probe.py	Transition probe training
src/evaluate_router.py	Residual scoring, calibration, and metric computation
src/visualize_pipeline.py	Live Rerun visualization pipeline
src/intervention_engine.py	Counterfactual synthetic repair generator
src/semantic_engine.py	Rule-based semantic scene-graph validator
src/train_downstream_detector.py	Downstream ablation study
results/singleview/	Single-view plots and evaluation outputs
results/multiview/	Multi-view evaluation outputs
results/downstream_ablation_results.csv	Final downstream ablation table
results/downstream_value_of_lift.png	Final downstream bar chart

Appendix B: Suggested Figures to Include

The following figures are recommended for the final PDF version:

1. Single-view calibration curve
2. Single-view Lift@k curve
3. Bootstrap histograms
4. Multi-view spatial anomaly screenshots
5. Rerun dashboard screenshot with:
 - overhead image
 - ego image
 - intervention view
 - hardware telemetry
 - temporal drift trace
 - semantic mismatch trace
6. Downstream Value of Lift bar chart

Example figure insertion:

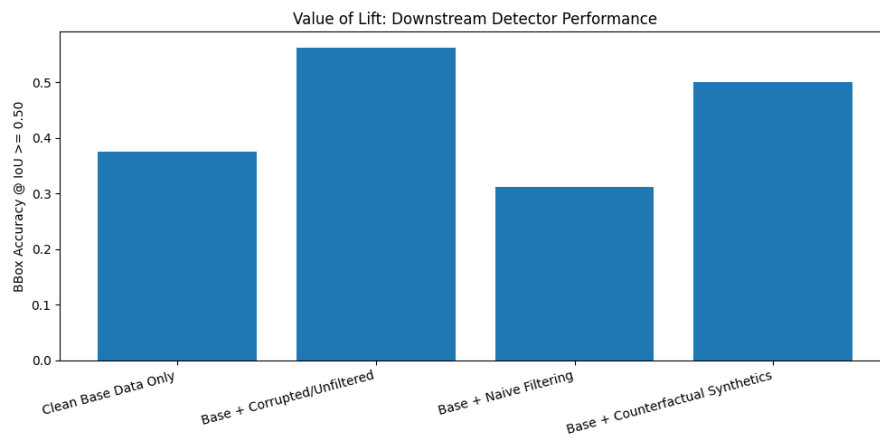


Figure 1: Downstream ablation study comparing four training configurations.